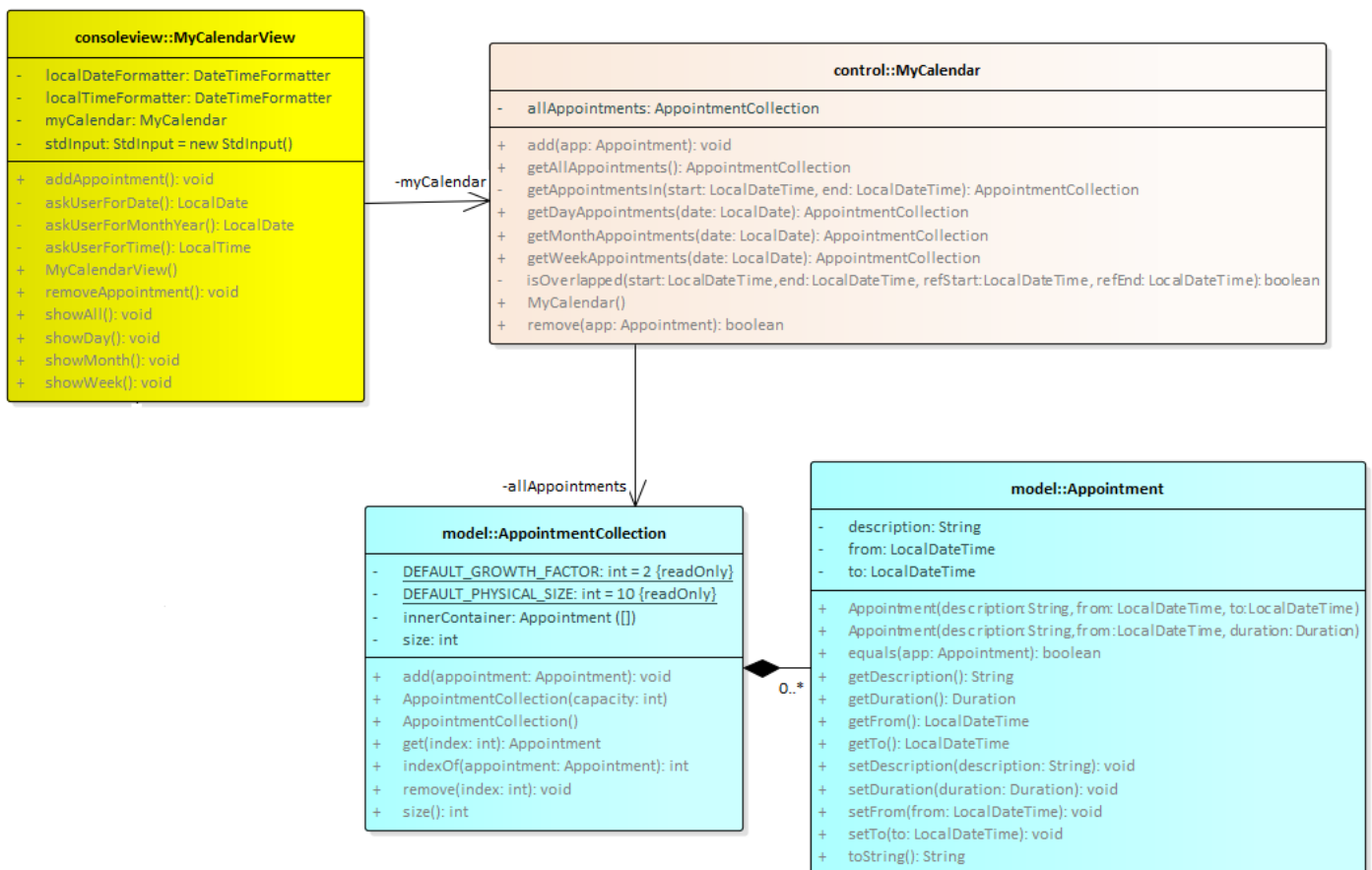


LAB – 07 – MyCalendar: filosofia della soluzione

Il problema dato è risolvibile mediante l'interazione di più entità. Si tratta della prima esercitazione che prevede una strutturazione complessa, pertanto è normale incontrare alcune difficoltà. Si consiglia di procedere con la lettura del presente documento tenendo sott'occhio il codice della soluzione.

Per comprendere bene la soluzione proposta, occorre rivedere attentamente le *slide* proposte in aula, specialmente quelle che riguardano il pattern *Model View Controller*. Come tutti i pattern, si tratta “solamente” di uno schema di soluzione, pertanto va opportunamente adattato al problema. Nel caso specifico, si prevede di strutturare la soluzione in tre parti distinte:

- **Model:** le classi `Appointment` e `AppointmentCollection` che rappresentano, rispettivamente, un appuntamento da memorizzare nel calendario e un insieme di appuntamenti.
- **Controller:** la classe `MyCalendar` che agisce sui dati prevedendo la logica necessaria per rispondere alle interazioni dell'utente.
- **View:** la classe `MyCalendarView` che, a sua volta, aggrega le funzionalità di varie altre classi e che realizza la parte di interfaccia utente.



Si noti come, giustamente, non sia necessario conoscere i dettagli di `MyCalendarView`: la parte da realizzare viene usata da tale classe ma non ha dipendenze verso di essa tanto che, sostituire la sola parte di interfaccia utente a console offerta da `MyCalendarView` con un altro tipo di interfaccia utente, non è certo un compito che stravolga la logica della soluzione (basta conoscere una opportuna tecnologia per creare interfacce utente).

In aula è stato proposto di procedere con l'implementazione partendo dalle classi che non hanno il minor numero di dipendenze (il cui funzionamento non dipende direttamente da altre classi del sistema): in questo modo è possibile procedere alla costruzione del “castello” partendo dalle fondamenta.

La prima classe da implementare, seguendo questo disegno, è Appointment. Tale classe rappresenta un singolo appuntamento. La sua implementazione non dovrebbe offrire particolari difficoltà, ma è opportuno notare due caratteristiche.

La prima è che Appointment è un'entità **modificabile**, quindi dotata di “trasformatori” che ne modificano lo stato; un appuntamento è un'entità che, nel modo reale, cambia al cambiare delle nostre necessità: è pratica comune “spostare” un appuntamento o cambiarne la descrizione. In questi termini un appuntamento non può essere modellato da un'entità immutabile poiché proprio l'entità corrispondente del mondo reale non lo è.

La seconda caratteristica è puramente algoritmica ed è relativa all'impostazione della durata dell'appuntamento: questo problema sarà affrontato successivamente.

model::Appointment
- description: String - from: LocalDateTime - to: LocalDateTime
+ Appointment(description: String, from: LocalDateTime, to: LocalDateTime) + Appointment(description: String, from: LocalDateTime, duration: Duration) + equals(app: Appointment): boolean + getDescription(): String + getDuration(): Duration + getFrom(): LocalDateTime + getTo(): LocalDateTime + setDescription(description: String): void + setDuration(duration: Duration): void + setFrom(from: LocalDateTime): void + setTo(to: LocalDateTime): void + toString(): String

Appointment rappresenta un appuntamento modellato come istante di inizio, istante di fine (java.time.LocalDateTime), descrizione e durata (java.time.Duration); questa è l'interfaccia pubblica e non è detto che corrisponda 1:1 con lo stato interno, infatti si sceglie di memorizzare istante di inizio, istante di fine e descrizione e di **calcolare la durata quando necessario**. Pertanto, i metodi getDuration e setDuration **non agiscono su un campo specifico ma calcolano ogni volta il risultato**; in particolare, getDuration calcola la durata partendo da istante di fine e istante di inizio dell'appuntamento, mentre setDuration imposta l'istante di fine in modo relativo all'istante di inizio al fine di ottenere la durata desiderata. Evitando di memorizzare anche l'informazione relativa alla durata, si evita l'onere di tenere sincronizzate tutte le informazioni: una soluzione che preveda semplicemente i campi inizio, fine e durata sui quali battono i metodi set/get corrispondenti, purtroppo non è corretta poiché, senza opportuni controlli, sarebbe possibile impostare un appuntamento che, ad es., va dalle 10:00 alle 12:00 con durata di 4 ore; la soluzione che prevede di calcolare la durata rende impossibile questa situazione.

```
public Duration getDuration() {  
    return Duration.between(from, to);  
}  
  
public void setDuration(Duration duration) {  
    to = from.plus(duration);  
}
```

Si noti come, grazie alla java.time API sia molto semplice calcolare la durata partendo da data/ora di inizio e fine, o impostare data/ora di fine partendo da data/ora di inizio e durata. Cosa ancora più importante è che i clienti di Appointment ignorano completamente il fatto che la durata dell'appuntamento sia calcolata e, più in generale, ignorano completamente l'effettiva implementazione di Appointment stesso (incapsulamento); vista da fuori, la soluzione che prevede di memorizzare istante di inizio e durata, sarebbe stata funzionalmente equivalente oltre che altrettanto corretta.

La classe AppointmentCollection incapsula un array che permette la memorizzazione di vari appuntamenti ed è analoga (in realtà, identica) a quella sviluppata per un laboratorio precedente.

model::AppointmentCollection	
-	<u>DEFAULT_GROWTH_FACTOR: int = 2 {readOnly}</u>
-	<u>DEFAULT_PHYSICAL_SIZE: int = 10 {readOnly}</u>
-	innerContainer: Appointment ([])
-	size: int
+	add(appointment: Appointment): void
+	AppointmentCollection(capacity: int)
+	AppointmentCollection()
+	get(index: int): Appointment
+	indexOf(appointment: Appointment): int
+	remove(index: int): void
+	size(): int

La classe MyCalendar incapsula una collezione di appuntamenti che viene creata nel costruttore; tale collezione contiene tutti gli appuntamenti gestiti dall'applicazione. I metodi di aggiunta e rimozione si limitano a delegare il funzionamento ai corrispondenti metodi della collezione di appuntamenti (il metodo remove deve prima individuare l'appuntamento da eliminare mediante il metodo indexOf della collezione, poi eliminarlo con removeAt).

control::MyCalendar	
-	allAppointments: AppointmentCollection
+	add(app: Appointment): void
+	getAllAppointments(): AppointmentCollection
-	getAppointmentsIn(start: LocalDateTime, end: LocalDateTime): AppointmentCollection
+	getDayAppointments(date: LocalDate): AppointmentCollection
+	getMonthAppointments(date: LocalDate): AppointmentCollection
+	getWeekAppointments(date: LocalDate): AppointmentCollection
-	isOverlapped(start: LocalDateTime, end: LocalDateTime, refStart: LocalDateTime, refEnd: LocalDateTime): boolean
+	MyCalendar()
+	remove(app: Appointment): boolean

La difficoltà maggiore nell'implementazione di tale classe si incontra quando si implementano i metodi di filtraggio. La soluzione procede in due passi:

1. si individua l'intervallo di tempo di interesse (un giorno, una settimana, ecc.);
2. si cercano gli appuntamenti che sono in qualche modo sovrapposti a tale intervallo di tempo.

Si noti come il secondo passo sia uguale per tutti i tipi di filtraggio e possa essere fattorizzato mediante un opportuno metodo (privato, AppointmentCollection getAppointmentsIn(LocalDateTime start, LocalDateTime end)).

L'intervallo di filtraggio è chiuso a sinistra e aperto a destra nello stesso modo in cui lo sono gli intervalli che rappresentano gli appuntamenti: un appuntamento che comincia alle 11:00 e termina alle 13:00, al minuto zero delle 13 è già finito quindi l'ultimo secondo utile dell'appuntamento è quello delle 12:59:59. Detto questo, individuare l'intervallo di filtraggio è piuttosto semplice.

Filtraggio per giorno: dato l'oggetto `LocalDate` che identifica il giorno di interesse, per creare gli estremi dell'intervallo di filtraggio basta creare come estremo inferiore un `LocalDateTime` partendo da quella `LocalDate` e impostando come orario le 00:00 e come estremo superiore un `LocalDateTime` ottenuto da quello appena creato dopo aver sommato un giorno (metodo `plusDays`).

```
public AppointmentCollection getDayAppointments(LocalDate date) {
    LocalDateTime dayStart = LocalDateTime.of(date, LocalTime.of(0, 0));
    LocalDateTime dayEnd = dayStart.plusDays(1);
    return getAppointmentsIn(dayStart, dayEnd);
}
```

Filtraggio per mese: dato l'oggetto `LocalDate` che identifica un qualsiasi giorno all'interno del mese di interesse, prima di tutto occorre creare il `LocalDate` che rappresenta il primo giorno del mese (basta ottenere una nuova `LocalDate` usando il metodo `withDayOfMonth` e passando 1), poi si possono ottenere l'estremo inferiore partendo dal `LocalDate` appena creato e impostando come orario le 00:00 e l'estremo superiore partendo dall'estremo inferiore e sommando un mese (metodo `plusMonths`).

```
public AppointmentCollection getMonthAppointments(LocalDate date) {
    LocalDate firstDayOfMonth = date.withDayOfMonth(1);
    LocalDateTime monthStart =
        LocalDateTime.of(firstDayOfMonth, LocalTime.of(0, 0));
    LocalDateTime monthEnd = monthStart.plusMonths(1);
    return getAppointmentsIn(monthStart, monthEnd);
}
```

Filtraggio per settimana: dato l'oggetto `LocalDate` che identifica un qualsiasi giorno all'interno della settimana di interesse, prima di tutto occorre creare il `LocalDate` che rappresenta il primo giorno della settimana, quindi il lunedì (per fare ciò, si calcola la distanza in giorni dal giorno corrente al lunedì precedente e si sottrae tale distanza dal `LocalDate` di partenza), poi si possono ottenere l'estremo inferiore partendo dal `LocalDate` appena creato e impostando come orario le 00:00 e l'estremo superiore partendo dall'estremo inferiore e sommando una settimana (metodo `plusWeeks`).

```
public AppointmentCollection getWeekAppointments(LocalDate date) {
    long distanceFromPastMonday =
        date.getDayOfWeek().getValue() - DayOfWeek.MONDAY.getValue();
    LocalDate weekStartDate = date.minusDays(distanceFromPastMonday);
    LocalDateTime weekStart =
        LocalDateTime.of(weekStartDate, LocalTime.of(0, 0));
    LocalDateTime weekEnd = weekStart.plusWeeks(1);

    return getAppointmentsIn(weekStart, weekEnd);
}
```

Ora l'operazione di filtraggio è relativamente semplice e non è legata in alcun modo alla natura dell'intervallo di interesse. In particolare, nella soluzione proposta l'operazione di filtraggio è fattorizzata nei due metodi riportati successivamente.

```
private AppointmentCollection getAppointmentsIn(Date start, Date end) {
    AppointmentCollection appList = new
        AppointmentCollection(allAppointments.size());
    for (int i = 0; i < allAppointments.size(); i++) {
        Appointment app = allAppointments.get(i);
        if (isOverlapped(app.getFrom(), app.getTo(), start, end)) {
            appList.add(app);
        }
    }
    return appList;
}

private boolean isOverlapped(LocalDateTime start, LocalDateTime end,
    LocalDateTime refStart, LocalDateTime refEnd) {
    return
        (start.isAfter(refStart) || start.isEqual(refStart)) &&
        start.isBefore(refEnd) ||
        (end.isAfter(refStart) || end.isEqual(refStart)) &&
        end.isBefore(refEnd) ||
        (start.isBefore(refStart) && end.isAfter(refStart));
}
```

O, semplificando (II versione):

```
private boolean isOverlapped(LocalDateTime start, LocalDateTime end,
    LocalDateTime refStart, LocalDateTime refEnd) {
    return start.isBefore(refEnd) && end.isAfter(refStart);
}
```

Naturalmente, in entrambi i casi, le precondizioni di utilizzo di `isOverlapped` sono:

- `start < end`
- `refStart < refEnd`

Il metodo `getAppointmentsIn` prende in ingresso un istante di inizio e un istante di fine per il filtraggio, predispone una nuova `AppointmentCollection`, poi scandisce tutti gli appuntamenti della collezione; ogni appuntamento che è in qualche modo sovrapposto all'intervallo di interesse (metodo `isOverlapped`) viene aggiunto alla collezione che sarà poi restituita. Si noti che il metodo `isOverlapped` racchiude la logica effettiva di filtraggio; al fine del funzionamento dell'algoritmo, tale metodo non è effettivamente necessario ma contribuisce a rendere il codice più leggibile e gestibile.

Roberta Calegari & Ambra Molesini